paceval / **paceval**

Code | Issues | Pull requests | Actions | Projects | Security and quality | Insights

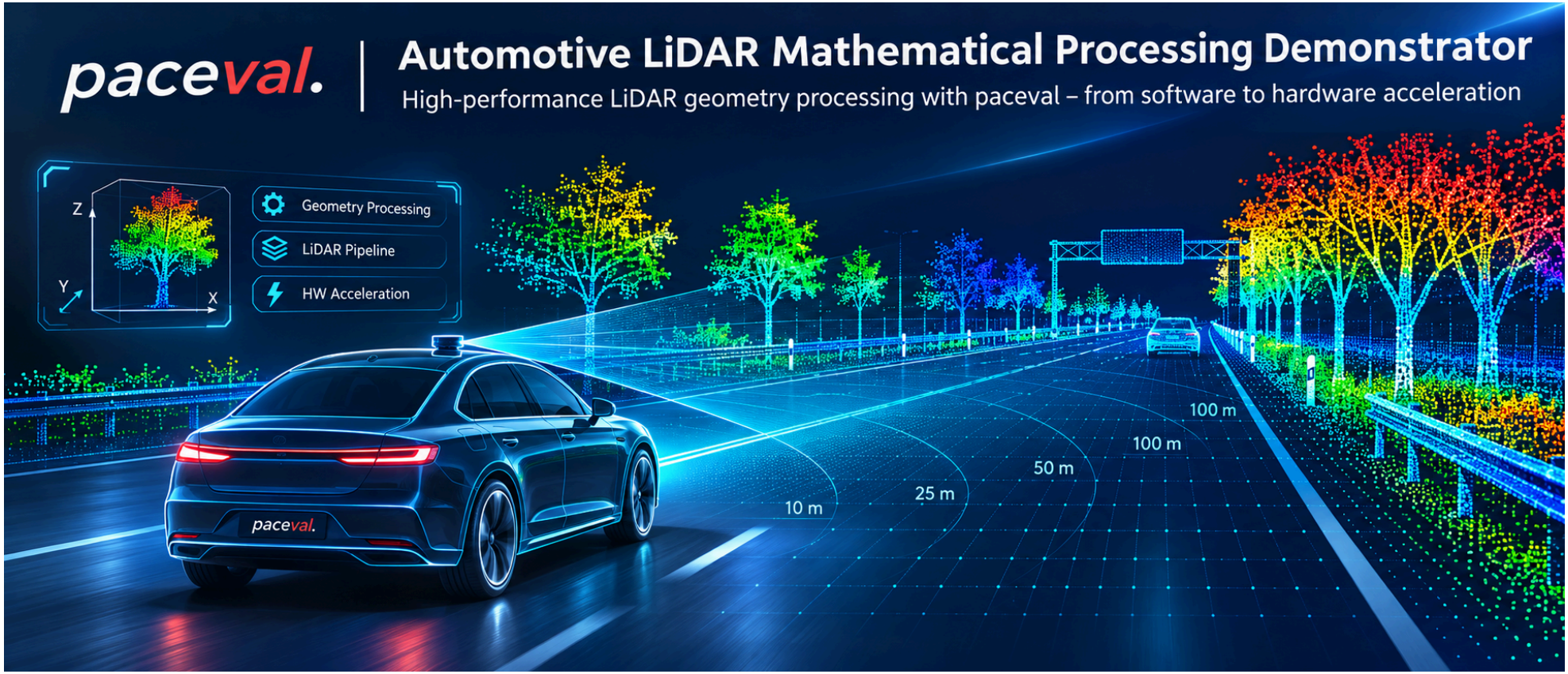
paceval / new projects / **Lidar with paceval** /

paceval Update README.md ✕ d1d9ce1 · 17 minutes ago History

Name	Name	Last commit date
..		
assets	LiDAR readme.md	1 hour ago
lidar_source_code (Windows x64)	LiDAR with paceval demonstrator	1 hour ago
README.md	Update README.md	17 minutes ago

README.md

paceval Automotive LiDAR Mathematical Processing Demonstrator



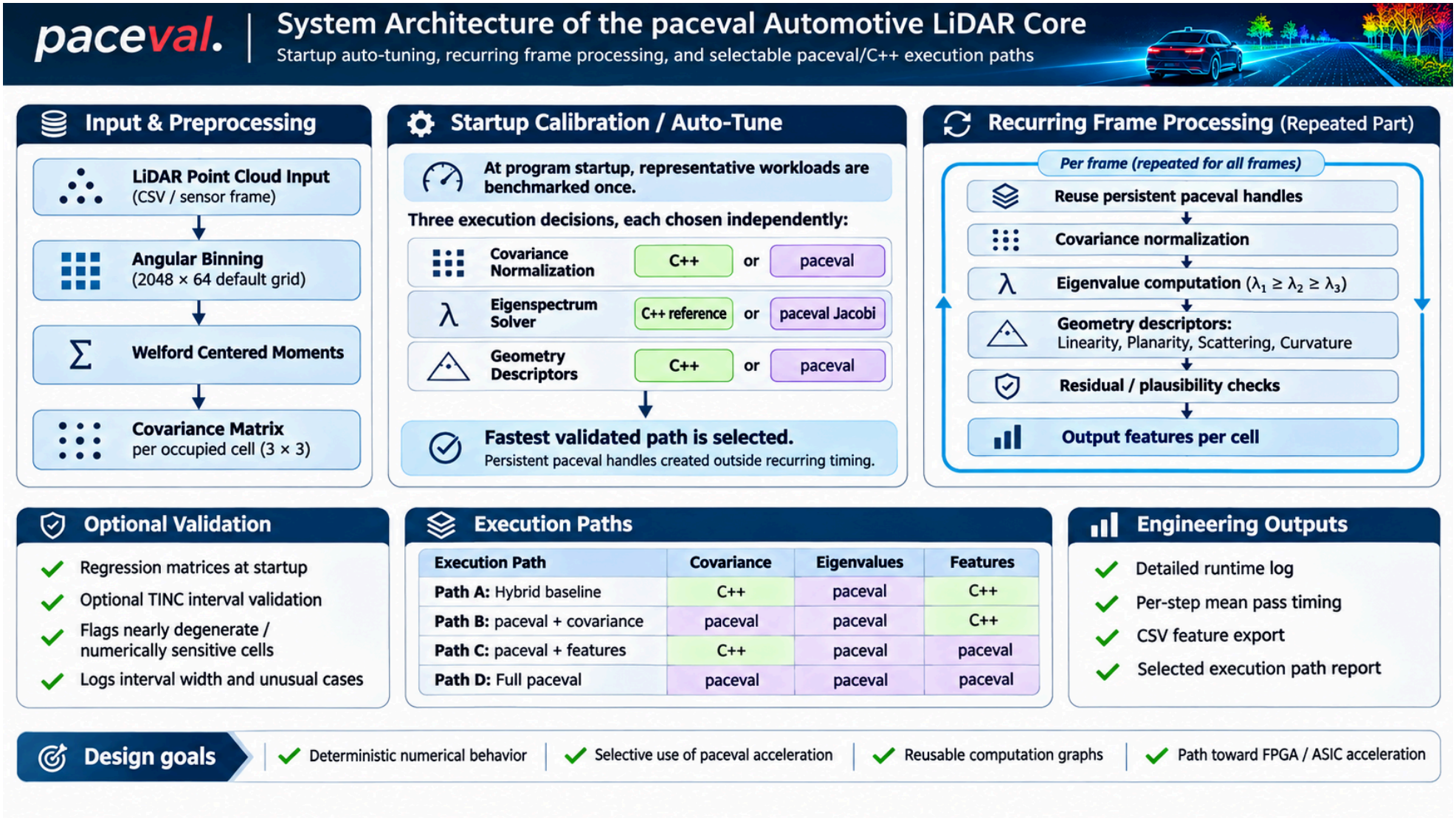
LiDAR and the role of paceval

LiDAR (Light Detection and Ranging) uses laser pulses to measure distances and build a three-dimensional point cloud of the surrounding environment. In vehicles, LiDAR can help perception systems detect and localize objects, understand road geometry, and support driver-assistance or automated-driving functions alongside other sensors.

The **paceval Automotive LiDAR Mathematical Processing Demonstrator** focuses on the numerical processing behind such systems, **not on complete object recognition or vehicle control**. It groups incoming points into angular cells, accumulates numerically stable Welford moments, derives 3 × 3 covariance matrices, calculates their eigenvalues, and extracts geometric descriptors such as linearity, planarity, scattering, and curvature. These features help characterize local point-cloud structures and can serve as inputs to downstream perception algorithms.

Available as a **Microsoft Visual C++ project** for Windows, the demonstrator compares four validated C++/paceval execution paths at startup and selects the fastest for the workload. Persistent paceval computation handles are reused across frames, while analytical regression tests, residual checks, and optional TINC interval diagnostics support numerical validation. Its broader objective is to evaluate how reusable mathematical computation graphs can move from CPU execution toward FPGA and ASIC acceleration.

System architecture



The architecture separates one-time calibration from recurring frame processing. This allows paceval to accelerate selected mathematical stages without requiring the entire LiDAR pipeline to run through the engine. It also creates a practical basis for evaluating larger, combined computation graphs on future hardware.

Measured performance

The reference measurements below were obtained on **Linux**, using an optimized GCC build and the original paceval shared library on an **AMD EPYC 9V74** environment (5 CPU cores visible to the container). The test frame contained **192,000 points**, **24,000 eligible cells**, and a **2048 × 64** angular grid. These are measured Linux reference values, **not Windows Visual C++ timings**.

Recurring processing: measured mean times per pass

Processing stage	Mean (ms)
Angular binning and Welford moments	17.271
Covariance normalization	0.194
Hybrid eigenspectrum computation	5.043
Geometry descriptors and residual checks	1.265
Total recurring computation	**23.773**
Math-only subset	6.502

Validation completed with **15/15 analytical regression cases passing**, no unconverged or bad-quality cells, no residual failures, and just **2 selective Jacobi fallbacks** in the tested frame. The recurring timings exclude startup calibration and computation-handle creation.

Outlook: paceval in hardware

The target is an **integrated FPGA streaming architecture with pipelined, reusable paceval execution graphs**. The engineering target is **20× faster execution of suitable mathematical graphs** (with **10–30×** as the exploratory design range). Equally important is moving beyond isolated function calls: combining covariance, eigenspectrum, and geometry-feature computations into fewer graph executions can reduce repeated host orchestration and support high-throughput processing of LiDAR cells.

Target scenario: integrated FPGA streaming and combined paceval graphs

Angular binning and Welford accumulation account for 73% of the measured CPU frame time. In an integrated design, dedicated FPGA streaming logic can handle this stage while persistent paceval graphs execute the subsequent mathematical processing. The objective is to keep data on-device, overlap processing where possible, and avoid repeated CPU–FPGA transfers.

For illustration, the model below retains **20× faster mathematical stages**, assumes **4× faster binning/Welford**, and adds **0.75 ms per frame** for remaining transfer and control costs:

Performance metric	Measured CPU	Integrated FPGA target
Binning and Welford (ms)	17.271	4.318
Combined paceval math stages (ms)	6.502	0.325
Transfer and control (ms)	—	0.750
Frame time (ms)	**23.773**	**5.393**
Frame rate (frames/s)	**42**	**185**
End-to-end speedup	**1×**	**4.41×**

This integrated design targets **approximately 4.4× higher end-to-end throughput**, with opportunities for further gains through additional parallel processing lanes and an optimized memory architecture. The projection is based on the measured CPU reference workload and the stated FPGA design assumptions.

A successful prototype would demonstrate not only higher frame throughput but also **lower latency and energy per processed cell**, establishing a scalable path from a programmable FPGA demonstrator toward a dedicated **paceval ASIC** for LiDAR and other numerically intensive applications.

The project is a **mathematical processing demonstrator**, not a production perception stack or safety-certified automotive ECU.

Built with **paceval®** · paceval.com · info@paceval.com

Copyright © 2015–2026 paceval.® All rights reserved.